

Metryco Laboratorio — Resumen de cambios

Migración PHP → React/Vite + Node/Express + MongoDB — sesión del 21 de agosto de 2026

Este documento describe, en orden, todo lo que se hizo en esta sesión: para qué sirve cada cambio y por qué se tomó esa decisión. Sirve como referencia si en el futuro hay que retomar, revisar o explicar este trabajo.

1. Auditoría inicial

Se revisó por completo el proyecto `metryco-laboratorio` (frontend nuevo, React + Vite) y el sistema `sistema.metrycolab.com` (PHP en producción, raíz + carpeta `/v2`).

Hallazgo clave: el frontend nuevo tenía 17 pantallas construidas visualmente, pero el 100% de los datos eran mock hardcodeado y no existía ningún backend. El sistema PHP tiene al menos 17 módulos de negocio; el frontend nuevo solo cubría 9 de ellos (visualmente) y no contemplaba 8, incluyendo Facturación Electrónica (CFDI/SAT), Técnicos/Asignaciones, Almacén, Proveedores/Compras, Portada de certificados, Plan de Producción y notificaciones automáticas.

Se generó una auditoría completa (matriz de comparación PHP vs. frontend nuevo, inventario de 17 módulos, arquitectura propuesta) publicada como documento aparte para consulta continua.

Decisiones confirmadas contigo

Tema	Decisión
Pantalla "General"	Se rehará para replicar el comportamiento del PHP viejo: listado general de usuarios de la empresa (lo que existe hoy en <code>General.jsx</code> era solo una prueba).
Facturación Electrónica (CFDI/SAT)	Fuera de alcance de esta migración — se mantiene tal cual está en el sistema PHP actual.
Almacén / Proveedores / Plan de Producción	No se ven en uso actualmente. Baja prioridad , quedan fuera del roadmap inicial con posibilidad de retomarlos si se necesitan.
Técnicos / Asignaciones	Confirmado que sí hay que implementarlo en el frontend nuevo. Es el módulo más grande y complejo del PHP (79 KB en un solo archivo) — pendiente ver el flujo real en producción antes de diseñar su modelo de datos.
Roles del sistema	<code>admin</code> , <code>tecnico</code> , <code>ventas</code> , <code>coordinador</code> (el frontend tenía <code>calidad</code> en vez de <code>coordinador</code> — corregido).
Notificaciones automáticas de vencimiento	Recomendación aplicada: job automático (<code>node-cron</code> , pendiente de implementar cuando se aborde ese módulo) + endpoint manual reutilizando el mismo servicio — mismo patrón que el botón manual del PHP actual.

2. Corrección de roles en el frontend

MODIFICADO Se corrigió el rol `calidad` (incorrecto) por `coordinador` (real), que es el que confirmaste.

- `metryco-laboratorio/src/shared/constants/roles.js` — `CALIDAD: "calidad" → COORDINADOR: "coordinador"`.
- `metryco-laboratorio/src/pages/Usuarios/Usuarios.jsx` — el mapa de colores/etiquetas de rol (`ROL_MAP`) actualizado igual.

3. Limpieza: carpeta de backend

Ya existía una carpeta `metryco-backend` con un scaffold de **Express + Prisma + PostgreSQL** (no era un repositorio git, sin forma de saber si era trabajo real o una prueba descartable — por eso no se tocó de entrada). Confirmaste que era una prueba y que el requisito real sigue siendo MongoDB.

ELIMINADO Se borró por completo la carpeta original `metryco-backend` (Prisma/PostgreSQL, incluyendo su `.env` con credenciales de una base local que no se usaba).

CREADO → RENOMBRADO Se creó un proyecto nuevo `metryco-api` (Express + Mongoose) y, una vez confirmado, se renombró a `metryco-backend` para quedarnos con un solo nombre de carpeta.

4. Backend nuevo: `metryco-backend`

Proyecto completo desde cero en `C:\Users\User\Desktop\metryco\metryco-backend`. Stack: Express + Mongoose + JWT (con roles) + bcrypt.

Estructura de carpetas

```
metryco-backend/
  src/
    config/      env.js (valida variables de entorno), db.js (conexión Mongo)
    models/      Usuario.js, Cliente.js
    services/     auth.service.js, usuario.service.js, cliente.service.js
    controllers/  auth.controller.js, usuario.controller.js, cliente.controller.js
    routes/       auth.routes.js, usuario.routes.js, cliente.routes.js, index.js
    middleware/   auth.js (verifica JWT), requireRole.js, errorHandler.js, notFound.js
    utils/        AppError.js, asyncHandler.js
    app.js        configuración de Express (cors, json, cookies, rutas, errores)
    server.js     punto de entrada: conecta Mongo y levanta el servidor
  scripts/
    seedAdmin.js crea el primer usuario admin desde variables de entorno
  .env.example
  README.md
```

Qué hace cada capa

Capa	Responsabilidad
<code>routes</code>	Solo mapea verbo HTTP + URL a un controller. Sin lógica.
<code>controllers</code>	Interpreta el request/response (body, params, status codes). Sin lógica de negocio.
<code>services</code>	Toda la lógica de negocio. Es la única capa que le habla a los <code>models</code> .
<code>models</code>	Schemas de Mongoose — la forma real de los datos en MongoDB.
<code>middleware</code>	Autenticación (JWT), autorización por rol, manejo centralizado de errores.

Módulos implementados

Auth — `POST /api/auth/login`, `POST /api/auth/refresh`, `POST /api/auth/logout`. El access token (JWT) dura 1h por defecto y lleva `{ id, usuario, rol }`; el refresh token vive en una cookie `httpOnly` (7 días) para que no sea accesible desde JavaScript en el navegador.

Usuarios — CRUD completo en `/api/usuarios`, protegido para rol `admin` únicamente. Contraseñas con hash `bcrypt`, nunca se guardan ni se devuelven en texto plano.

Clientes — CRUD completo en `/api/clientes`, protegido para cualquier usuario autenticado. Campos calcados de los que ya pedía `ClienteForm.jsx` del frontend (fiscales, domicilio, contacto, facturación). El "eliminar" es un soft-delete (marca `status: inactivo`) en vez de borrar el registro, porque un cliente normalmente está referenciado por cotizaciones/reportes/cobranza.

Variables de entorno requeridas (`.env` , no incluido en el repo)

```
PORT=4000
MONGODB_URI=mongodb://... (local o Atlas)
JWT_SECRET=...
JWT_EXPIRES_IN=1h
JWT_REFRESH_SECRET=...
JWT_REFRESH_EXPIRES_IN=7d
CORS_ORIGIN=http://localhost:5173
SEED_ADMIN_USUARIO=admin
SEED_ADMIN_PASSWORD=...
SEED_ADMIN_NOMBRE=Administrador
SEED_ADMIN_EMAIL=admin@metrycolab.com
```

Actualización: ya se conectó a tu MongoDB local y se probó de punta a punta — ver sección 6.

Verificación realizada

- `npm install` corrió sin errores (134 paquetes, 0 vulnerabilidades).
- Todos los archivos `.js` pasaron `node --check` (sintaxis válida).
- Se verificó que el servidor **rechaza arrancar** si faltan variables de entorno obligatorias (protección real, no decorativa).

5. Frontend: conexión a la API real (Login + Clientes)

Antes de este cambio, el login era un usuario/contraseña hardcodeado en el código (`admin/admin`) y todas las pantallas usaban arreglos de datos ficticios. Se conectaron **Login** y el módulo **Clientes** de punta a punta contra `metryco-backend` . El resto de los módulos (Usuarios, Cotizaciones, Reportes, etc.) se quedan con datos mock por ahora — se conectan uno por uno según el orden de migración acordado.

Archivo	Cambio
<code>.env</code> / <code>.env.example</code> (nuevo, raíz del frontend)	NUEVO <code>VITE_API_URL=http://localhost:4000/api</code>
<code>.gitignore</code>	MODIFICADO se agregó <code>.env</code> para no versionarlo
<code>src/services/api.js</code>	REESCRITO instancia real de Axios con <code>baseUrl</code> , interceptor que adjunta el token JWT a cada petición, e interceptor de 401 que intenta refrescar el token una vez antes de mandar a <code>/login</code>
<code>src/core/auth/LoginService.js</code>	REESCRITO ya no hay usuario/contraseña hardcodeados; llama a <code>POST /api/auth/login</code>
<code>src/core/auth/AuthContext.js</code>	NUEVO el <code>Context</code> de auth se separó de <code>AuthProvider.jsx</code> a su propio archivo (corrige un warning real de Fast Refresh que ya existía antes de esta sesión)
<code>src/core/auth/AuthProvider.jsx</code>	MODIFICADO ahora importa el contexto desde el archivo nuevo; <code>logout()</code> también avisa al backend para invalidar la cookie de refresh
<code>src/services/clientes.js</code>	NUEVO funciones <code>listarClientes</code> , <code>obtenerCliente</code> , <code>crearCliente</code> , <code>actualizarCliente</code> , <code>eliminarCliente</code>
<code>src/shared/hooks/useDebounce.js</code>	IMPLEMENTADO estaba vacío; ahora existe y se usa para no disparar una búsqueda en cada tecleo del buscador de clientes
<code>src/pages/Clientes/ClientesPage.jsx</code>	REESCRITO ya no usa el arreglo <code>MOCK</code> ; trae datos reales de la API con búsqueda, filtro por sector y paginación en el servidor; agrega estados de carga y error visibles
<code>src/pages/Clientes/ClienteForm.jsx</code>	REESCRITO cliente real: si es edición, trae los datos del cliente por su id; al guardar, crea o actualiza contra la API; muestra el error del servidor si algo falla

Bug real encontrado y corregido

En `ClienteForm.jsx` , los campos "Email Cotizaciones" y "Email Facturación" estaban registrados con **el mismo nombre de campo** (`register("email")` en ambos), así que el segundo pisaba

silenciosamente el valor del primero — nunca se hubiera guardado el correo de cotizaciones. Se corrigió usando nombres distintos que además coinciden con el modelo del backend (`contacto.emailCotizaciones` / `contacto.emailFacturacion`).

Por qué los nombres de campo del formulario cambiaron

El formulario de cliente ahora registra los campos con notación de punto (`domicilioFiscal.calle` , `contacto.telefono` , `facturacion.formaPago` , etc.) en vez de nombres planos. React Hook Form arma automáticamente el objeto anidado a partir de esos nombres, y esa forma anidada es exactamente la que espera el modelo `cliente` de Mongoose — así se evita tener que transformar el objeto a mano antes de enviarlo.

Un ajuste de código por una regla de lint nueva y estricta

Detalle técnico, solo relevante si se vuelve a tocar este archivo: el proyecto usa una versión muy reciente de `eslint-plugin-react-hooks` (v7, reglas del "React Compiler") que prohíbe llamar, desde un `useEffect` , a una función aparte que internamente actualice estado — aunque sea asíncrona. La solución fue poner la lógica de carga de datos directamente dentro del `useEffect` (función `async` autoejecutada) en vez de llamar a una función `cargar()` separada, y usar el patrón que React recomienda para "reiniciar una página al cambiar un filtro" sin pasar por un efecto.

Verificación realizada

- `eslint` sobre todos los archivos tocados: sin errores.
- `npm run build` del frontend completo: compiló correctamente.

6. Conexión a MongoDB y prueba real de punta a punta

Diste la connection string de tu MongoDB local: `mongodb://localhost:27017/`, base `metryco`. Se completó el `.env` de `metryco-backend` con esa URI y con secretos JWT generados aleatoriamente (48 bytes cada uno).

- `npm run seed:admin` corrió correctamente y creó el usuario admin real (usuario `admin`, contraseña temporal `Mettryco2026!` — pendiente cambiarla en cuanto exista pantalla de perfil).
- Se levantaron ambos servidores (`metryco-backend` en :4000, `metryco-laboratorio` en :5173) y se probó el login real por HTTP: el token JWT y la cookie de refresh se emiten correctamente.
- Confirmaste que el login también funciona desde el navegador.

Bug de seguridad real encontrado y corregido durante las pruebas

Al probar la creación de usuarios por API, el endpoint `POST /api/usuarios` devolvía el **hash de la contraseña** (bcrypt) en la respuesta. La causa: la opción `select: false` del schema de Mongoose solo excluye el campo en consultas (`find`, `findById`), pero **no** en el documento que devuelve `.create()`. Se corrigió agregando una transformación `toJSON` en `Usuario.js` que elimina `passwordHash` de cualquier respuesta, sin importar por qué camino se generó el documento — defensa a nivel de modelo, no solo por endpoint.

7. Módulo Usuarios conectado a la API real

Antes, "Nuevo Usuario" y "Editar Usuario" eran formularios HTML sueltos sin `onSubmit` funcional (no guardaban nada). Se reescribieron por completo:

Archivo	Cambio
<code>src/services/usuarios.js</code>	IMPLEMENTADO antes solo tenía un comentario; ahora tiene <code>listarUsuarios</code> , <code>obtenerUsuario</code> , <code>crearUsuario</code> , <code>actualizarUsuario</code> , <code>desactivarUsuario</code>
<code>src/shared/components/NuevoUsuario/NuevoUsuario.jsx</code>	REESCRITO de HTML crudo sin funcionalidad a un formulario real (React Hook Form + MUI) que crea el usuario contra la API, con validación y manejo de errores (p. ej. usuario duplicado)
<code>src/shared/components/EditarUsuario/EditarUsuario.jsx</code>	REESCRITO igual que el anterior; la contraseña es opcional al editar (se deja en blanco para no cambiarla)
<code>src/pages/Usuarios/Usuarios.jsx</code>	REESCRITO ya no usa el arreglo <code>MOCK</code> ; lista usuarios reales con búsqueda y

paginación en el servidor; "Desactivar" ahora sí desactiva al usuario (soft-delete) con diálogo de confirmación

Nota: el campo "Firma" que tenía el formulario original del PHP (subida de imagen) se dejó fuera por ahora — el backend todavía no tiene un mecanismo de subida de archivos (multer, almacenamiento en disco/S3, etc.). Es una decisión pendiente, no un olvido: hay que definir dónde se van a guardar los archivos (firmas, y más adelante PDFs de certificados) antes de construirlo, para no hacerlo dos veces.

Verificación realizada

- Se probó por API real: crear usuario, listar, y rechazo correcto (409) al intentar crear un usuario con el mismo `usuario` ya existente.
- `eslint` sobre todos los archivos nuevos/modificados: sin errores.
- Los usuarios de prueba creados durante la verificación se borraron de la base — solo queda el usuario admin real.

8. Módulo Cotizaciones conectado a la API real

Cotizaciones es el primer módulo con lógica de negocio propia (folio consecutivo, cálculo de IVA, referencia a un cliente real) — no es un CRUD tan simple como Clientes o Usuarios.

Backend

Archivo	Qué hace
<code>src/models/Counter.js</code>	NUEVO colección auxiliar para generar folios consecutivos de forma atómica (evita folios duplicados si dos personas cotizan al mismo tiempo)
<code>src/models/Cotizacion.js</code>	NUEVO folio, referencia al cliente (<code>ObjectId</code>), partidas, subtotal/IVA/total, estado, quién la creó
<code>src/services/cotizacion.service.js</code>	NUEVO genera el folio (<code>COT-2026-001</code> , consecutivo por año), calcula subtotal/IVA (16%)/total en el servidor a partir de las partidas — nunca confía en totales que mande el navegador — y arma la búsqueda por folio o nombre de cliente usando <code>\$lookup</code> de Mongo (el cliente vive en otra colección)
<code>src/controllers/cotizacion.controller.js</code> , <code>src/routes/cotizacion.routes.js</code>	NUEVO CRUD completo en <code>/api/cotizaciones</code> , protegido para cualquier usuario autenticado

Frontend

Archivo	Cambio
<code>src/services/cotizaciones.js</code>	NUEVO funciones para listar/obtener/crear/actualizar/eliminar
<code>src/pages/Cotizaciones/CotizacionesPage.jsx</code>	REESCRITO ya no usa el arreglo <code>MOCK</code> ; datos reales con búsqueda, filtro por estado y paginación en el servidor
<code>src/pages/Cotizaciones/CotizacionForm.jsx</code>	REESCRITO el selector de "Cliente" ahora carga la lista real de clientes (ya no son 5 nombres fijos); si no hay clientes registrados, se avisa antes de dejar cotizar; al guardar, el total en pantalla es solo una vista previa — el IVA y el total oficial siempre se calculan en el servidor

Verificación realizada

- Prueba real por API: se creó un cliente, dos cotizaciones seguidas (folios `COT-2026-001` y `COT-2026-002` , consecutivos), se buscó por nombre de cliente, se editaron las partidas de una

cotización (el total se recalculó solo) y se eliminó.

- `eslint` y `npm run build` del frontend: sin errores nuevos.
- Datos de prueba borrados de la base al terminar (incluyendo el contador de folios, para que el primero real que genere el sistema sea `COT-2026-001`).

9. Qué falta / próximos pasos

1. Cambiar la contraseña temporal del admin (Metryco2026!) en cuanto exista una pantalla para hacerlo.
 2. Definir la estrategia de almacenamiento de archivos (firmas de usuario, y más adelante PDFs de certificados/cotizaciones) antes de construir esa parte — Cotizaciones ya está lista para eso pero por ahora no genera PDF ni envía correo, como sí hace el PHP.
 3. Ver el flujo real de **Técnicos/Asignaciones** en el sistema en producción antes de diseñar ese módulo (es el más grande y crítico, no se debe adivinar).
 4. Seguir con el orden de migración acordado: Reportes → Calidad/Equipos/Patronos/Actividades/Cobranza → Técnicos/Asignaciones.
-

Generado automáticamente al cierre de la sesión del 21 de agosto de 2026. Para el detalle completo de la auditoría (inventario de los 17 módulos del sistema PHP y matriz de comparación), ver el documento de auditoría publicado por separado.